

I2C COMMUNICATION PROTOCOL

Digital IC Design Using FPGA

RTL Implementation and Verification Report

National Telecommunication Institute (NTI)

Team Members

Ahmed Shaaban Elnegaar	297553
Hazim Osama shams elden	226358
Mohamed Abdel Sadek Abdel Hakem	170265

1. Introduction

Modern digital systems often require multiple integrated circuits (ICs) to exchange information efficiently. Parallel communication can require many interconnecting wires, while serial communication reduces the number of required signal lines. The Inter-Integrated Circuit (I2C) protocol provides a simple synchronous serial interface using two shared lines: SDA for bidirectional data and SCL for clock synchronization.

This project presents the design and implementation of an I2C communication system in Verilog HDL. The merged design consists of an I2C Master, an I2C Slave, a clock-divider function, a shared open-drain bus, finite-state-machine controllers, and a verification testbench. The design supports both write and read transactions and includes monitoring/debugging signals that make internal FSM states, bit counters, data, busy status, and error status visible during simulation.

The objective of this project is to design and implement a fully functional I²C communication system, including both the Master and Slave modules, and to verify its operation using a self checking testbench that covers the possible communication scenarios. This report first explains the operation of the I²C protocol. It then presents a detailed breakdown of the RTL implementation for each module and discusses the verification methodology used to validate the design.

2. Project Overview

I2C, often called I two C, stands for the Inter-Integrated Circuit protocol. I2C was invented in 1982 by Philips Semiconductor (now NXP Semiconductor) as a low speed communication protocol for connecting microprocessor master devices with lower-speed peripheral slave devices. Since 2006, implementing the I2C protocol does not require a license, and many semiconductor device companies.

The objective of this project is to design and implement a complete I2C communication system consisting of a single I2C Master connected to a shared two-wire bus.

Although the I2C protocol supports communication with multiple slave devices on the same bus, this implementation includes a single I2C Slave for verification purposes.

The master controls the entire communication process by generating the serial clock (SCL), issuing the START and STOP conditions, transmitting the slave address, and managing all read and write transactions.

The slave continuously monitors the shared bus and only responds when its assigned address is detected.

The system communicates over two shared bus lines:

- SCL: Serial Clock Line generated by the master.
- SDA: Bidirectional Serial Data line shared by all devices.

The implemented design supports two communication modes:

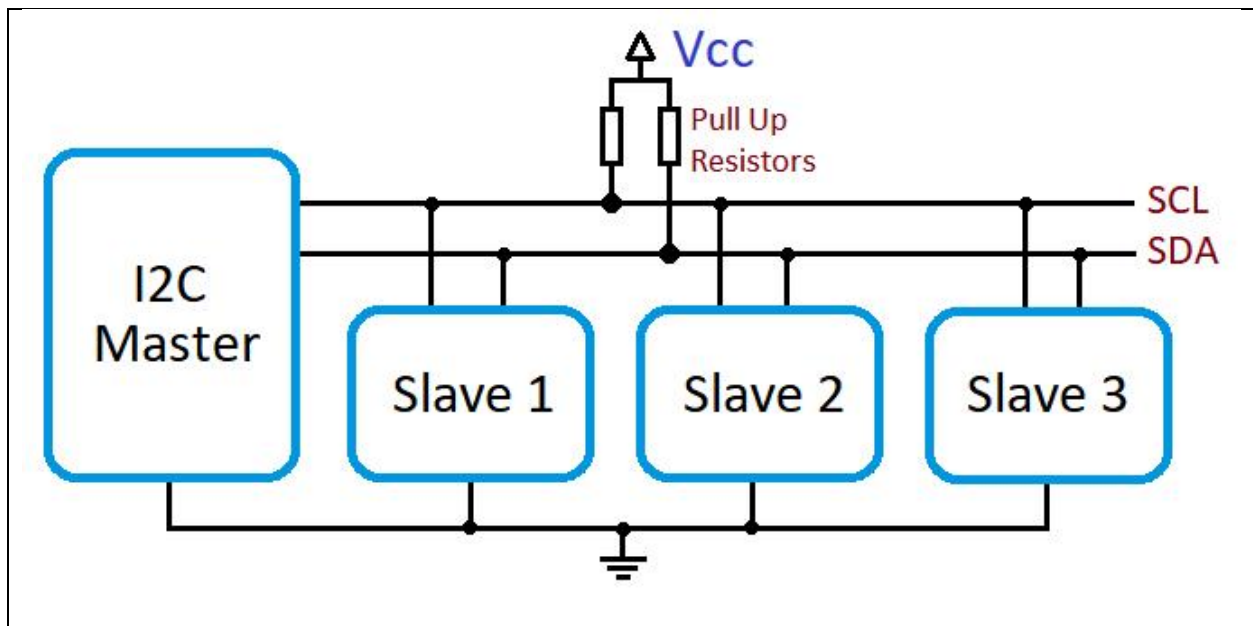
- Write Operation: Master → Slave
- Read Operation: Slave → Master

Each transaction transfers a single 8-bit data byte between the master and the addressed slave, with acknowledgement (ACK) used after both the address and data bytes to ensure reliable communication.

3. I²C Protocol Fundamentals

3.1 Bus Lines

- Two bidirectional lines are used: SDA and SCL.
- Both lines use open-drain/open-collector behavior with external pull-up resistors so the idle bus is HIGH.
- The protocol supports multiple devices on the same bus. The supplied protocol description identifies a 400 pF capacitance limit and a 128-address space.



3.2 Transmission Rules

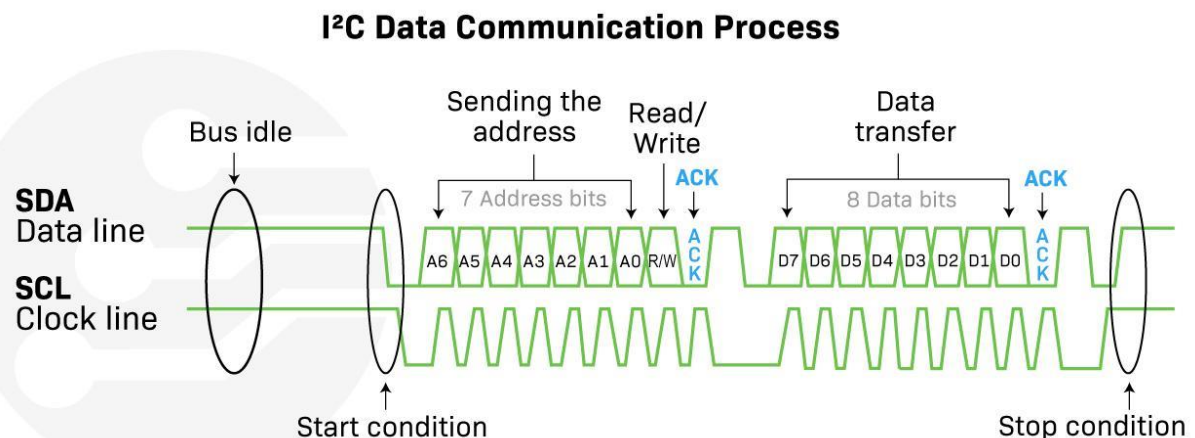
- Data is transferred in 8-bit bytes.
- Bits are transmitted MSB first.
- SDA changes while SCL is LOW and remains stable while SCL is HIGH, except for START and STOP conditions.
- Standard-mode operation is 100 kbit/s; the project implementation targets 100 kHz SCL.
- Each transmitted byte is followed by an ACK or NACK clock cycle.

3.3 START and STOP Conditions

- START: SDA transitions from HIGH to LOW while SCL is HIGH.
- STOP: SDA transitions from LOW to HIGH while SCL is HIGH.
- These transitions mark the beginning and end of an I²C transaction.

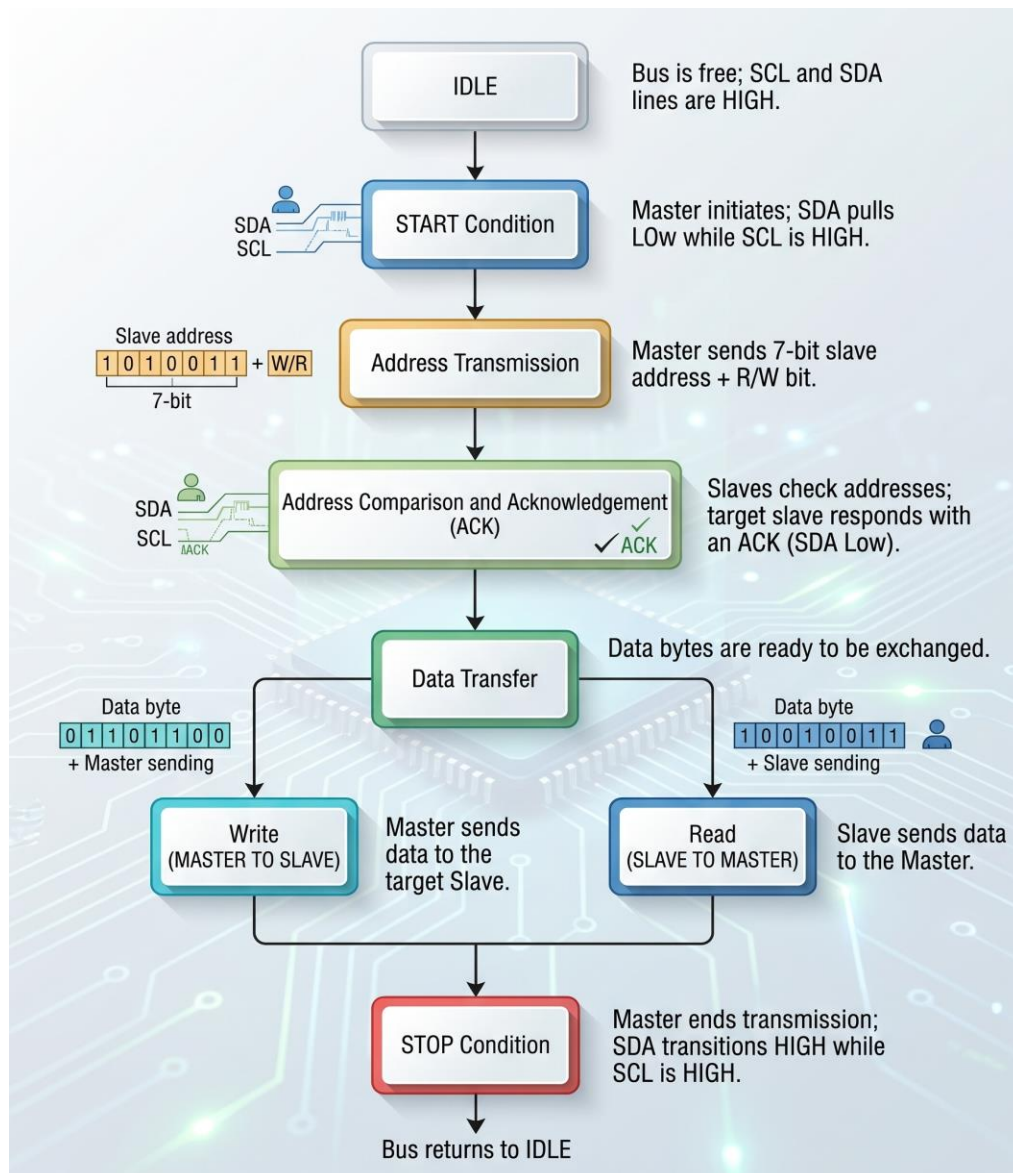
3.4 Address Byte and ACK/NACK

The address byte contains a 7-bit slave address followed by the Read/Write control bit. A receiver acknowledges by pulling SDA LOW during the ninth clock pulse. If SDA remains HIGH, the receiver has not acknowledged the byte (NACK), which may indicate an error or the end of a read transfer.



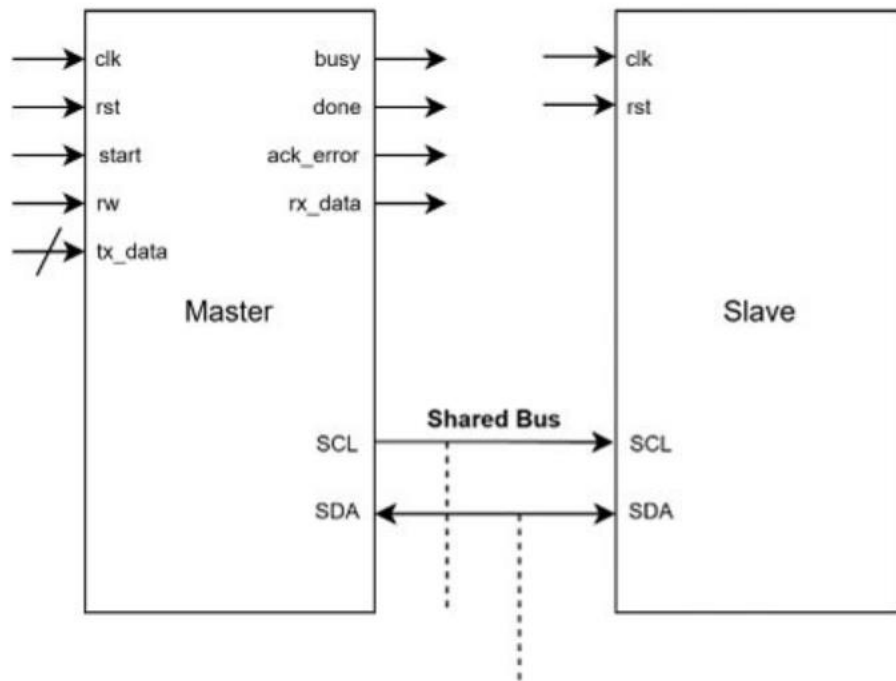
4. Communication Sequence

1. Idle: SCL and SDA are HIGH and no device is transmitting.
2. START: the Master pulls SDA LOW while SCL remains HIGH.
3. Address transmission: the Master sends the 7-bit address followed by the R/W bit.
4. Address comparison: the Slave compares the received address with its configured address.
5. Address ACK: a matching Slave pulls SDA LOW during the ACK cycle.
6. Data transfer:
7. If Write : The Master sends the data bytes.
8. If Read : The Slave takes control of SDA to send the data bytes, while the Master continues generating the SCL clock pulses.
9. STOP: the Master releases SDA HIGH while SCL is HIGH, returning the bus to idle.



5. System Architecture

The top-level design contains the Master controller, Slave controller, clock-divider function and shared SDA/SCL bus. The Master initiates transactions and controls SCL, while the Slave monitors the bus and responds only when its address is detected.



6. Finite State Machine Design

Both the Master and Slave controllers are implemented as Moore finite state machines. The state-driven structure provides predictable sequencing and separates transaction phases into clear, testable states.

6.1 Master FSM

State	Function
IDLE	Waits for start; SDA/SCL are released and busy is LOW.
START_ST	Generates START and prepares the address/RW information.
SEND_ADDR	Serially transmits the 7-bit address and R/W bit.
ADDR_ACK	Releases SDA and waits for the Slave ACK.
WRITE_DATA	Serially transmits the write byte.
DATA_ACK	Waits for the Slave to acknowledge the data byte.
READ_DATA	Releases SDA and samples the Slave data on SCL rising edges.
MASTER_ACK	Completes the read acknowledgement phase before STOP.
STOP_LOW	Prepares the STOP sequence by driving SDA LOW while SCL is LOW.
STOP_HIGH	Releases SDA while SCL is HIGH to generate STOP.
DONE_ST	Pulses done for one system-clock cycle, then returns to IDLE.

6.2 Slave FSM

State	Function
IDLE	Monitors the bus for a valid START condition.
ADDRESS	Receives the 7-bit slave address from SDA, one bit per detected SCL rising edge, and compares it with the configured address.
R_W	Receives the Read/Write bit after the 7-bit address. Stores it in r_or_w to determine whether the Master wants to read or write.
ACK_ADDR	Generates an ACK if the received address matches the configured slave address.
DATA	Handles the 8-bit data transfer.
ACK_DATA	Generates the data ACK after the 8-bit data transfer and checks whether another transaction or STOP condition follows.
STOP	Waits for a valid STOP condition on the I ² C bus, then returns to IDLE.

7.1 I²C Master

The Master generates START and STOP conditions, transmits the address and R/W bit, performs write/read transfers, checks ACK responses, and reports transaction status through busy, done and ack_error signals.

Port	Direction	Width	Description
clk	Input	1	FPGA system clock.
Rst_n	Input	1	Active-lasynchronous reset.
start	Input	1	Begins a new I ² C transaction.
rw	Input	1	Selects write (0) or read (1).
tx_data	Input	8	Byte transmitted during write operations.
scl_in	Input	1	Current state of the SCL bus.
sda_in	Input	1	Current state of the SDA bus.
scl_drive_low	Output	1	Drives SCL low or releases it.
sda_drive_low	Output	1	Drives SDA low or releases it.
busy	Output	1	Indicates an active transaction.
done	Output	1	Pulses when the transaction completes.
ack_error	Output	1	Indicates that an ACK was not received.
rx_data	Output	8	Byte received during read operations.

7.2 I²C Slave

The Slave continuously monitors SDA and SCL for a START condition, receives and compares the address, generates ACK when addressed, receives data during writes, and transmits stored data during reads.

Port	Direction	Width	Description
clk	Input	1	FPGA system clock.
rst	Input	1	Asynchronous reset.
Scl_s_in	Input	1	Serial clock from the bus.
Sda_s_in	Input	1	Current state of the SDA bus.
sda_drive_low	Output	1	Drives SDA low or releases it.

7.3 Clock Divider

The project uses a 50 MHz FPGA system clock and generates a 100 kHz I²C SCL clock for Standard-mode operation. The divider counts system-clock cycles and toggles the SCL output after a predefined count. The Slave does not need its own divider because it observes the SCL generated by the Master.

Port	Direction	Width	Description
Clk_in	Input	1	FPGA system clock.
rst	Input	1	Asynchronous reset.
Clk_out	Output	1	Generated I ² C serial clock.

```
`default_nettype none

module clk_div100k #(parameter num = 250)(
    input wire  clk_in,
    input wire  rstn,
    output reg  clk_out
);
localparam width = $clog2(num);
reg [width-1:0] counter;

always @(posedge clk_in or negedge rstn)begin
    if (!rstn) begin
        counter <= {width{1'b0}};
        clk_out <= 1'b0;
    end else begin
        if (counter <= num-1 )begin
            counter <= 1'b0;
            clk_out <= ~clk_out;
        end else begin
            counter <= counter + 1'd1;
        end
    end
end
endmodule
```

7.4 Shared Open-Drain Bus

- A device can drive SDA LOW or release the line to high impedance (Z).
- No device actively drives SDA HIGH.
- When all devices release SDA, the external pull-up resistor makes the bus HIGH.
- The same principle is used for SCL in the implementation's bus interface.

8. Timing and Synchronization

Reliable I²C operation depends on the transmitter changing SDA while SCL is LOW and the receiver sampling SDA while SCL is HIGH. This separation allows each transmitted bit to stabilize before it is sampled. In the implementation, the Master uses the generated SCL timing while the Slave reacts to the SCL edges present on the shared bus.

10. Verification Methodology

The verification environment uses a self-checking Verilog testbench. It instantiates the top-level I²C system, generates clock and reset signals, performs write and read transactions, monitors status outputs, and reports transaction results. Reusable tasks are used for reset, write and read operations.

10.1 Test Cases

Test Case	Stimulus	Expected Result
Reset	Apply asynchronous reset before communication.	Registers initialize and Master/Slave enter IDLE.
Single Write	Write 0xA5 from Master to Slave.	Slave acknowledges and stores 0xA5.
Multiple Write	Write 0x00, 0xFF, 0x55 and 0xAA sequentially.	Each transaction completes and the Slave stores the transmitted byte.
Read Operation	Master performs a read transaction.	Slave transmits stored data and Master reconstructs the byte.
Random Write	Write ten randomly generated bytes.	Transactions complete without acknowledgement errors.
Invalid/Wrong Address	Address a non-matching Slave ID.	Slave ignores the transaction and the Master detects an acknowledgement error.

10.2 Self-Checking Mechanism

- Write operations wait for the done signal before reporting completion.
- Read operations compare received data against the expected value.
- ack_error is monitored to detect missing acknowledgements.
- busy, done, ack_error, rx_data and slave receive data are displayed for debugging and waveform analysis.

11. Simulation Results and Waveforms

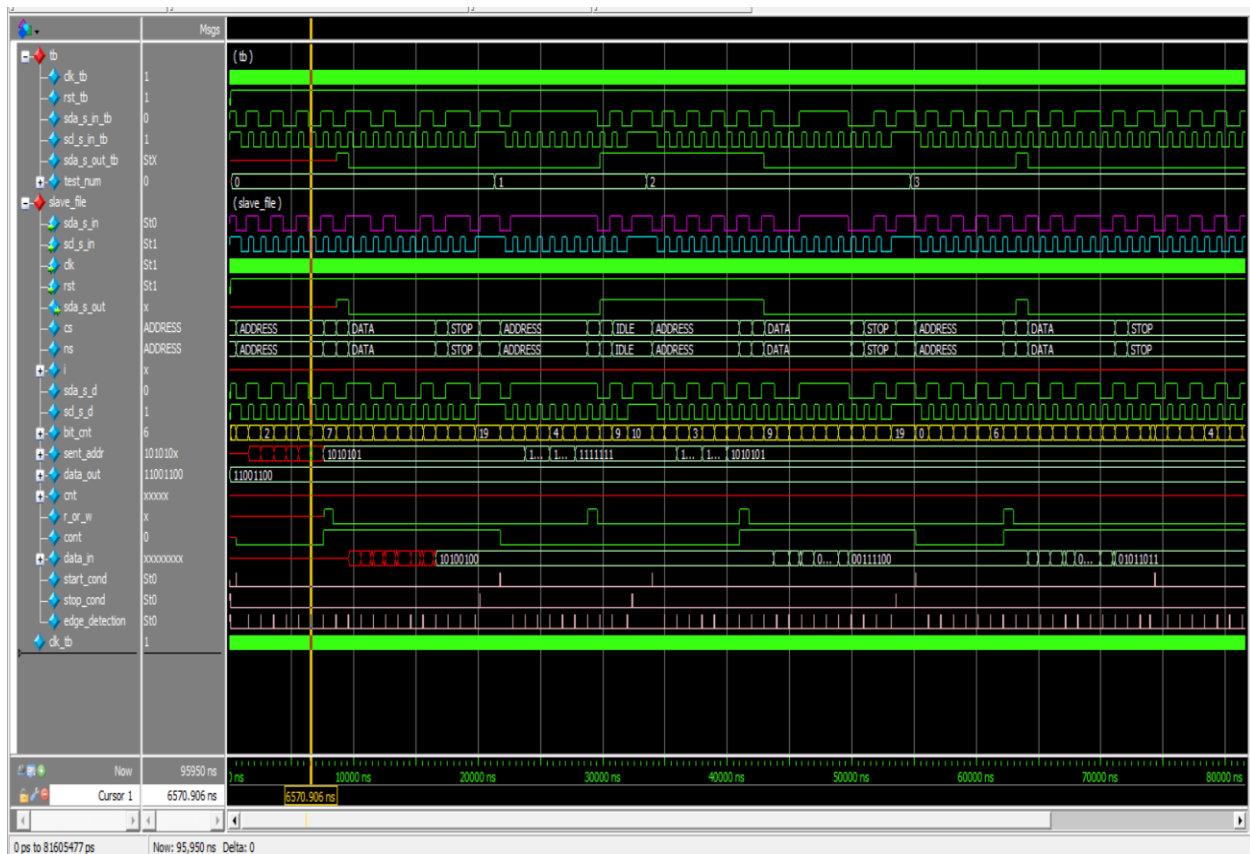
The supplied reports show successful simulation of write, read and wrong-address scenarios. The implementation report also includes a simulation setup in which the Master sends two data bytes and requests two data bytes from the Slave.

Master



```
# Write DATA = 10110110
# DONE = 1
# ACK ERROR = 0
# TEST 1 : PASS
# -----
# TEST 2 : Read
# READ DATA = 10110110
# ACK ERROR = 0
# TEST 2 : PASS
# -----
# TEST 3 : Write
# Write DATA = 11010110
# DONE = 1
# ACK ERROR = 0
# TEST 3 : PASS
# -----
# TEST 4 : Read
# READ DATA = 11010110
# ACK ERROR = 0
# TEST 4 : PASS
# -----
# TEST 5 : ADDRESS ACK ERROR
# DONE = 0
# ACK ERROR = 1
# TEST 5 : PASS
# -----
# TEST 6 : DATA ACK ERROR
# Write DATA = 11010110
# DONE = 0
# ACK ERROR = 1
# TEST 6 : PASS
# -----
# TEST 7 : Write
# Write DATA = 11110000
# DONE = 1
# ACK ERROR = 0
# TEST 7 : PASS
# -----
```


Slave:



Loading sv_std.std

Loading work.slave

Test 1 Passed with addr_ack=1 data_ack=1 at time 19350

Test 2 Passed (correctly NACKed wrong address) at time 31550

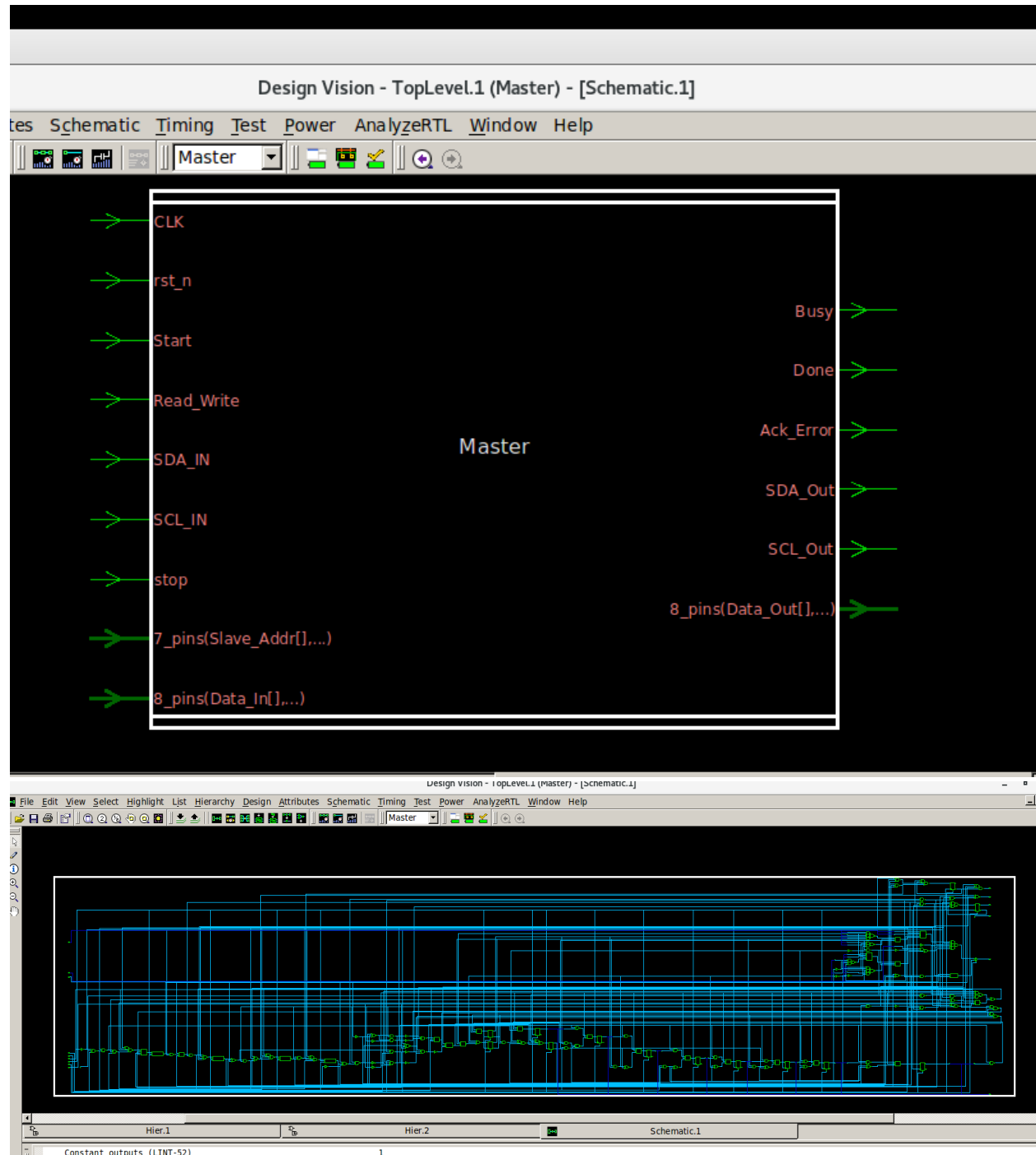
Test 3 Passed with addr_ack=1 data_ack=1 at time 52750

Test 4 Passed (two-frame transaction) at time 94350

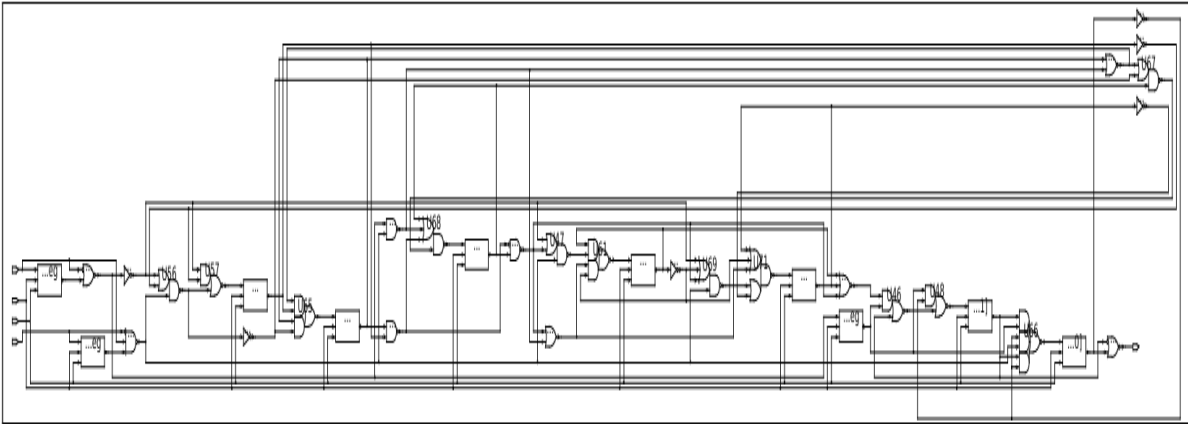
12. Schematic and Synthesis Results

The supplied implementation report contains elaborated and synthesized schematics for the Master and Slave. The second report also presents the synthesized top-level design and utilization information.

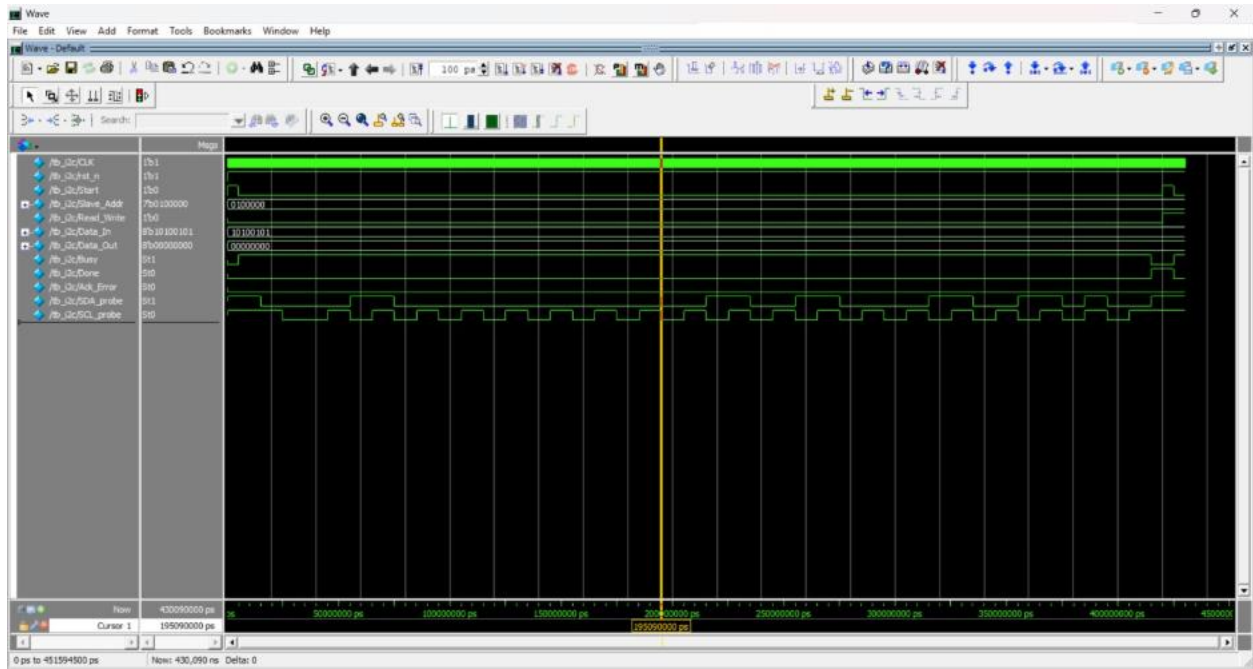
MASTER



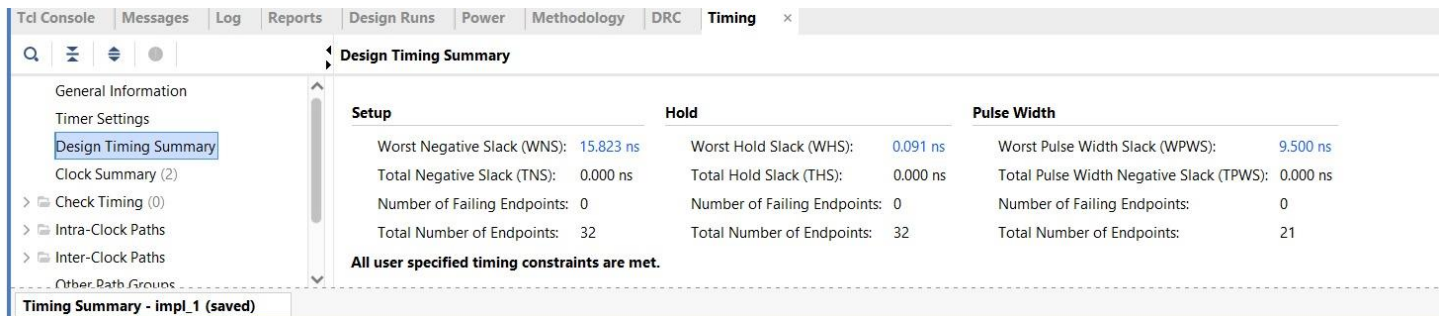
Slave



TB_Waveform:



13. Static Timing Analysis



The screenshot shows a software interface with a menu bar at the top containing 'Tcl Console', 'Messages', 'Log', 'Reports', 'Design Runs', 'Power', 'Methodology', 'DRC', and 'Timing'. The 'Timing' menu is open, showing a 'Design Timing Summary' report. On the left, a sidebar lists 'General Information', 'Timer Settings', 'Design Timing Summary' (highlighted), 'Clock Summary (2)', 'Check Timing (0)', 'Intra-Clock Paths', 'Inter-Clock Paths', and 'Other Path Groups'. The main area displays the 'Design Timing Summary' report with three columns: 'Setup', 'Hold', and 'Pulse Width'. The report shows various timing metrics and concludes with 'All user specified timing constraints are met.'.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 15.823 ns	Worst Hold Slack (WHS): 0.091 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 32	Total Number of Endpoints: 32	Total Number of Endpoints: 21
All user specified timing constraints are met.		

This Design Timing Summary report confirms that all user-specified timing constraints for the implemented digital design have been successfully met, with **zero failing endpoints** across setup, hold, and pulse width checks.

Setup Analysis:

- **Worst Negative Slack (WNS):** +16.370ns
- **Total Negative Slack (TNS):** 0.000ns
- **Failing Endpoints:** 0 out of 30 total endpoints.
- **Interpretation:** The positive WNS indicates that data paths arrive well before the clock edge requirements, providing a comfortable timing margin for the operating frequency.

Hold Analysis:

- **Worst Hold Slack (WHS):** +0.180 ns
- **Total Hold Slack (THS):** 0.000 ns
- **Failing Endpoints:** 0 out of 30 total endpoints.
- **Interpretation:** The positive WHS confirms that data remains stable long enough after the clock edge, ensuring there are no hold violations.

Pulse Width Analysis:

- **Worst Pulse Width Slack (WPWS):** +9.500 ns
- **Total Pulse Width Negative Slack (TPWS):** 0.000 ns
- **Failing Endpoints:** 0 out of 21 total endpoints.
- **Interpretation:** High pulse width slack guarantees that clock high/low time constraints are fully respected